GovSkill.

Sovereign Civic Platform & Document Intelligence

Master Technical Guide, Fresh Laptop Setup, Live Drizzle Studio & College Viva Preparation Dossier

GITHUB REPOSITORY LINK https://github.com/sachin-saroj/GovSkill	LIVE PRODUCTION FRONTEND https://govskill-frontend.onrender.com
LIVE PRODUCTION BACKEND HEALTH https://govskill-backend.onrender.com/health	INTERACTIVE SWAGGER API DOCS https://govskill-backend.onrender.com/docs
RENDER CLOUD DASHBOARD dashboard.render.com (GovSkill Project)	VISUAL DATABASE STUDIO (DRIZZLE) `open_db_studio.bat` • `https://local.drizzle.studio`

Executive Summary: GovSkill local municipal aur revenue administration offices ke liye design kiya gaya ek sovereign civic platform hai. Isme do core pillars hain: (1) **Internal Civil Servant Competency** — administrative curriculum reading, dual-mode grounded AI mentor, server-evaluated anti-tamper examinations, aur HMAC-SHA256 digitally signed certificates; (2) **GovAssist Citizen Pre-Submission Checker** — documents ka client-side OCR extraction, 100% deterministic 4-rule validation engine, aur verification QR code wala printable counter slip.

Section 1: Dost Ke Laptop Ka Setup Guide (Step-by-Step)

Agar aapke dost ke laptop par ye project shuru se chalana hai, to ye exact steps follow karein. Sab kuch simple aur tested hai.

Step 0: Prerequisites (Zaroori Software)

Software	Recommended Version	Kyu Chahiye?	Check Command
Git	2.35+	GitHub se code clone karne ke liye	`git --version`
Python	3.11, 3.12, ya 3.13	FastAPI backend aur database engine ke liye	`python --version`
Node.js & npm	Node 18+ ya 20+ LTS	React 18 frontend aur Vite dev server ke liye	`node -v` aur `npm -v`

Step 1: Clone the GitHub Repository

Dost ke laptop me Terminal ya PowerShell kholein (e.g., Desktop par) aur run karein:

```
git clone https://github.com/sachin-saroj/GovSkill.git
cd GovSkill
```

Step 2: Backend Setup (Virtual Environment & SQLite DB)

```
# 1. Backend folder me jayein
cd backend

# 2. Python ka isolated virtual environment banayein
python -m venv venv

# 3. Virtual environment activate karein
# Windows (PowerShell):
.□env\Scripts□ctivate
# Mac / Linux:
source venv/bin/activate

# 4. Backend dependencies install karein
pip install -r requirements.txt

# 5. Environment file banayein (.env.example se copy karein)
copy .env.example .env    # (Mac/Linux par: cp .env.example .env)

# 6. Database schema migrate karein (Isse govskill.db file turant ban jayegi)
alembic upgrade head
```

> **Point to Remember:** Koi PostgreSQL install karne ki zaroorat nahi hai! `alembic upgrade head` run karte hi local SQLite database file (`govskill.db`) automatically create ho jayegi jisme saari 8 migrations applied rahengi.

Step 3: Frontend Setup

Ek naya terminal window/tab kholein aur frontend dependencies install karein:

```
cd frontend
npm install
```

Step 4: Application Run Karna (2 Easy Methods)

Method A: 1-Click Launcher (Super Fast on Windows)

Project ke main folder me `start_dev.bat` file hai. **Bas uspe double-click kar do!**
Wo automatic Backend (`http://localhost:8000`) aur Frontend (`http://localhost:3000`) dono ko separate windows me start kar dega.

Method B: Manual Commands

- **Terminal 1 (Backend API):**

  ```
  cd backend
  .□env\Scripts□ctivate
  uvicorn app.main:app --reload --port 8000
  ```

- **Terminal 2 (Frontend Client):**

  ```
  cd frontend
  npm run dev
  ```

Step 5: Testing & Verification

- **Open Web Application:** http://localhost:3000
- **Login Page:** http://localhost:3000/login (Staff account banayein ya sign in karein — **instant 200ms** me login hoga).
- **Interactive Swagger API Docs:** http://localhost:8000/docs
- **Health Check:** http://localhost:8000/health → `{"status":"ok","app":"GovSkill","database":"connected"}`

Section 2: Database Deep-Dive & Live Drizzle Studio Setup

1. Database Kaha Se Ho Raha Hai? Kaise Ho Raha Hai? Data Save Kaha Ho Raha Hai?

Ye examiner ka favorite question hota hai. Isko samajhna bilkul simple hai:

Question	Exact Technical Answer
Data save kaha hota hai?	Local environment me sara data `backend/govskill.db` naam ki SQLite binary file ke andar save hota hai. Ye file SQLite B-Tree format me disk par rehti hai. Production (Render) par primary target PostgreSQL 16 tha, lekin fault-tolerance ke liye wahan bhi SQLite fallback automatic kaam karta hai.
Database connection kaise banta hai?	FastAPI direct SQL queries nahi likhta. Hum **SQLAlchemy 2.0 Async ORM** use karte hain (`app/db/session.py`). Driver `sqlite+aiosqlite:///./govskill.db` use hota hai jo async event loop ko bina block kiye queries chalata hai. Connection pooling me `pool_pre_ping=True` enabled hai.
Tables kaise banti hain?	Tables manually nahi banti. Python classes (`app/models/`) database tables ko represent karti hain. **Alembic** tool migrations execute karta hai (`alembic upgrade head`). Migration scripts 001 se 008 tak table structures, primary keys, aur columns create karti hain.
Jab PC band ho jaye ya server restart ho to data bachta hai?	Local PC par `govskill.db` file hard drive par permanently saved rehti hai, isliye data hamesha surakshit rehta hai. Render free tier par disk ephemeral (temporary) hoti hai, isliye cloud par restart hone par fresh state start hoti hai.

2. Live Visual Database Studio (Drizzle Studio Setup)

Project me humne **Drizzle Studio** setup kar diya hai, jisse aap poore database ko ek sundar web dashboard me live dekh aur edit kar sakte hain:

> **Drizzle Studio Ko Kaise Chalayein?**
>
> - **Option 1 (1-Click):** Project root me `open_db_studio.bat` par double-click karein.
> - **Option 2 (Terminal):** Root folder me run karein:
>
> ```
> npm run db:studio
> ```
>
> - Browser me open karein: **https://local.drizzle.studio**

Drizzle Studio Me Kya-Kya Live Dikhai Deta Hai?

- `users` — Registered employees aur admins, unke hashed passwords, email, role, aur age.
- `modules` — 4 default training modules aur unka administrative curriculum text.
- `user_progress` — Kis employee ne kaunsa module padha, status (completed/certified), aur best score.
- `quiz_questions` — Questions, options (JSON array), correct answers, aur competency tags.
- `quiz_attempts` — Har attempt ka score, timestamp, aur pass/fail status.
- `credentials` — Pass hone par issue huye cryptographic digital certificates, unique human-readable certificate IDs (`GS-CERT-YYYY-HEX`), aur unka HMAC-SHA256 signature digest.

- `citizen_documents` — Citizens dwara upload kiye gaye documents, extracted fields (Name, Certificate Number, Validity Date), aur validation status.

Section 3: Frontend, Backend & Database Connections (How Everything Works)

1. Frontend & Backend Ka Connection Kaise Hota Hai?

1. **Vite Development Proxy:** Local dev me frontend `http://localhost:3000` par chalta hai. `vite.config.ts` me `/api` ko `http://localhost:8000` par proxy kiya gaya hai. Isse CORS (Cross-Origin Resource Sharing) issue zero ho jata hai.
2. **Axios Client Instance (`frontend/src/lib/api.ts`):**
   - Centralized baseURL: `import.meta.env.VITE_API_BASE_URL || '/api'`.
   - **JWT Interceptor:** Har API call se pehle localStorage se token check karta hai: agar token hai to header me `Authorization: Bearer <token>` add kar deta hai.
   - **45s Timeout Guard:** Cloud cold starts ko handle karne ke liye timeout 45 seconds rakha gaya hai. Agar network error aaye to user-friendly alert show karta hai.

2. Core Workflows (Data Flow Diagram)

Workflow A: Citizen Document Pre-Validation Flow

```
Citizen File Upload (PDF/PNG/JPG) at /citizen
   │
   ▼
Frontend Axios POST multipart/form-data to /api/documents/upload
   │
   ▼
Backend Security: File size (< 5MB) + Whitelisted Extension (.pdf, .png, .jpg)
   │
   ▼
PyMuPDF renders PDF to Image ──► Pillow Preprocessing (Grayscale + 2.0x Contrast + Binarization)
   │
   ▼
Tesseract OCR extracts raw text ──► Regex extracts Name, Certificate No, Expiry Date
   │
   ▼
Deterministic 4-Rule Engine (100% Python Code):
   ├── Rule 1: Name Presence & Validity
   ├── Rule 2: Certificate Pattern Regex Format
   ├── Rule 3: Validity & Expiry Date Comparison
   └── Rule 4: Issuing Authority / Seal Presence
   │
   ├─► [ALL PASS] ──► Save in citizen_documents ──► Return Printable Counter Slip with QR Code
   └─► [FAIL] ─────► Gemini AI generates plain-language fix ──► Show remediation to Citizen
```

Workflow B: Employee Assessment & Digital Credential Minting Flow

```
Employee takes 8-Question MCQ Exam at /quiz
   │
   ▼
Backend GET /api/quiz/{id} (correct_option_index is STRIPPED to prevent cheating!)
   │
   ▼
Employee submits answers ──► POST /api/quiz/{id}/submit
   │
   ▼
Authoritative Server-Side Scoring against DB Answer Keys
   │
   ▼
Score >= 75% ?
   ├── YES ──► Mint HMAC-SHA256 Signed Credential (GS-CERT-YYYY-HEX) ──► Mark User as "Certified"
   └── NO  ──► Record Attempt ──► AI Tutor provides targeted remediation on weak areas
```

Section 4: 15 Deep Project Viva Questions & Answers (Hinglish)

Ye questions examiners GovSkill ke specific implementation par puchte hain. Inhe confidence ke sath explain karein:

Q1: GovSkill project kya hai aur iska real-world problem statement kya hai?

Answer: GovSkill local municipal aur revenue administration ke liye ek full-stack civic platform hai. Real world me do badi problems hoti hain: (1) Local government staff ke paas administrative knowledge to hoti hai lekin digital proficiency kam hoti hai, jisse verification me errors aate hain; (2) Citizens government counters par incomplete ya expired documents le aate hain, jisse 40% forms counter par hi reject ho jaate hain. GovSkill staff ko train karke certified karta hai aur citizens ko counter par aane se pehle document pre-check karne ki facility deta hai.

Q2: Document pass ya fail karne ke liye Gemini AI ka use kyun nahi kiya? Deterministic Python rule engine kyun banaya?

Answer (Sabse Important Question): Sir, **LLMs probabilistic hote hain aur hallucinate kar sakte hain**. Ek fake document prompt ki wajah se pass ho sakta hai ya ek genuine document reject ho sakta hai. Statutory aur legal decisions me hallucination ki koi jagah nahi hoti. Isliye hamara pass/fail decision **100% deterministic Python code** leta hai (regex format, expiry date math, issuing authority check). Gemini AI ko strictly **Explainer** ki tarah isolate kiya gaya hai — wo sirf un rules ka simple language me solution batata hai jo hamara Python code already fail declare kar chuka hai.

Q3: Django ya Flask chhodkar FastAPI kyun choose kiya?

Answer: FastAPI choose karne ke 3 solid reasons hain: (1) **High Asynchronous Concurrency:** FastAPI ASGI aur Python ke `async/await` event loop par chalta hai. Document OCR aur AI requests I/O-heavy hote hain; async handling se server multi-user requests ko bina worker threads block kiye fast process karta hai. (2) **Pydantic v2 Type Safety:** Har request payload automatically validate hota hai. (3) **Self-Documenting Swagger UI:** `/docs` par automatic interactive documentation milti hai.

Q4: Quiz me koi employee browser DevTools kholkar answer key chura sakta hai kya?

Answer: Bilkul nahi! Hamare architecture me **`correct_option_index` client ko kabhi transmit hi nahi hota** — use backend me strip kar diya jata hai. Client ke paas sirf question text aur option strings aate hain. Evaluation exclusively server-side `/api/quiz/{id}/submit` endpoint par database ke secret answer key se match karke hota hai.

Q5: Digital Credential kaise sign hota hai? Kya koi duplicate certificate bana sakta hai?

Answer: Duplicate banana impossible hai. Jab employee ≥ 75% score karta hai, backend user ID, module ID, score aur timestamp ko canonicalize karke **HMAC-SHA256** algorithm aur dedicated server-only `CREDENTIAL_SIGNING_KEY` se digitally sign karta hai. Verification page (`/verify/:credentialId`) par server signature recompute karta hai. Agar certificate ID, score ya name me ek character bhi badla gaya ho, to cryptographic signature mismatch ho jayega aur certificate fake declare ho jayega.

Q6: Dual-Mode AI Mentor kya hai aur AI ko syllabus ke bahar baat karne se kaise rokte ho?

Answer: Humne central `AIGateway` ke peeche do modes banaye hain: (1) **`GROUNDED_TRAINING` Mode:** Ye departmental circulars aur modules ke saath strictly bound rehta hai (`find_relevant_modules`). Agar user out-of-syllabus sawal puchega (jaise recipe ya movies), to AI politely refuse kar deta hai. (2) **`GENERAL_CHAT` Mode:** Ye open administrative doubts aur general questions ke liye multilingual copilot hai jo English, Hindi (Devanagari), aur Hinglish natural code-switching support karta hai.

Q7: Dual-database architecture (PostgreSQL + SQLite) kyun banaya?

Answer: Production cloud hosts (jaise Render free tier) par database expire ya suspend ho sakte hain. Humne **Resilient Dual-Dialect Engine** banaya hai. Container boot script (`entrypoint.sh`) pehle PostgreSQL check karta hai; agar database unreachable ya suspended ho, to wo automatically local SQLite fallback (`govskill.db`) activate kar deta hai. Isse viva, presentation ya production me service kabhi crash nahi hoti aur zero-downtime rehti hai.

Q8: Agar citizen mobile camera se aadhi-tedhi ya low-quality photo upload kare to OCR kaise padhta hai?

Answer: Direct OCR mobile photos par fail ho jata hai. Isliye humne **Pillow Preprocessing Pipeline** banayi hai: image ko 8-bit grayscale me convert karte hain, contrast 2.0x boost karte hain, aur adaptive threshold binarization se black-and-white contrast create karte hain. Agar local Tesseract confidence low deta hai, to pipeline automatically Gemini Vision OCR fallback trigger karti hai.

Q9: DPDP Act 2023 (Citizen Privacy) ka compliance kaise ensure kiya hai?

Answer: Teen tareeko se: (1) **Zero Citizen Registration:** Citizens bina account banaye documents check kar sakte hain, jisse unwanted demographic data collect hi nahi hota. (2) **PII Isolation:** `citizen_documents` table ka `users` table se koi foreign key relation nahi hai. (3) **Masking:** Aadhaar (`XXXX-XXXX-1234`) aur PAN (`ABCDE****F`) API payloads aur logs me permanently mask hote hain.

Q10: Rate limiting kaise implement ki hai aur IP spoofing kaise rokte ho?

Answer: Humne in-memory sliding-window limiter lagaya hai (Document upload par 20 req/min, login par 15 req/min). IP spoofing rokne ke liye `get_client_ip` function `X-Forwarded-For` header par sirf tabhi trust karta hai jab incoming connection trusted loopback reverse proxy (Docker/Render) se aa raha ho. Direct spoofed headers ignore ho jaate hain.

Q11: Database me Alembic ka kya use hai? Table direct code se kyun nahi banayi?

Answer: Code se direct table banane (jaise `create_all()`) me purana data delete hone ka risk rehta hai aur naye columns (jaise `age` column) add nahi ho paate. Alembic **version-controlled migrations** provide karta hai. Har change ek migration file ban kar record hota hai jise audit aur rollback kiya ja sakta hai.

Q12: React 18 me code-splitting kaise ki hai aur iska kya fayda hua?

Answer: Humne React ke `React.lazy()` aur `Suspense` ka use karke sabse heavy pages (`AdminDashboardPage` aur `CitizenUploadPage`) ko standalone async chunks me isolate kiya. Isse initial JavaScript bundle 734 kB se ghatkar 638 kB ho gaya (~13% faster page load on mobile).

Q13: Cloud cold-start issue ko frontend me kaise solve kiya?

Answer: Render free tier par 15 min baad container sleep ho jata hai aur boot hone me 30-40s leta hai. Pehle Axios timeout 15s tha to request time out ho jati thi. Humne timeout badha kar **45s** kiya aur `apiError.ts` me interceptor lagaya jo `ECONNABORTED` detect karke user ko friendly message deta hai: "*Server cold sleep se wake up ho raha hai, kripya thoda wait karein.*"

Q14: Project me automated testing ki kya coverage hai?

Answer: Hamare paas **100% passing automated test suite** hai: Backend me 119 Pytest tests (Auth, OCR fixtures, Rate limiting, HMAC credentials, Rule engine) aur Frontend me 121 Vitest tests across 23 test suites (UI components, accessibility checks via `jest-axe`, login forms). GitHub Actions CI pipeline har push par inko verify karta hai.

Q15: Agar 10,000 citizens ek sath certificate upload karein to system scale kaise karoge?

Answer: Scale karne ke liye: (1) OCR processing ko background queue (Celery/Redis) me offload karenge aur citizen ko instant `job_id` denge. (2) Stateless FastAPI containers ko Kubernetes ya AWS ECS par horizontally auto-scale karenge. (3) Database read-replicas lagayenge aur uploaded files ko local disk ke bajay AWS S3 ya GCP Cloud Storage par store karenge.

Section 5: 20 Technical Fundamentals Questions & Answers (Hinglish)

Ye questions fundamental computer science aur web technologies ke baare me hain jo kisi bhi technical round ya viva me basic concepts test karne ke liye puche jaate hain:

T1: Backend kya hota hai aur web application me iska kya role hai?

Answer: Backend kisi application ka brain hota hai jo server par chalta hai aur user ko direct nahi dikhta. Iska kaam business logic execute karna, database se data fetch/save karna, security aur authentication handle karna, aur frontend ko structured data (JSON) provide karna hota hai.

T2: Frontend kya hota hai aur user interface kaise kaam karta hai?

Answer: Frontend application ka wo hissa hai jise user browser me dekhta hai aur interact karta hai (buttons, forms, animations, layouts). Iska kaam user input lena, use backend API tak pahunchana, aur API se aane wale data ko sundar UI me display karna hota hai.

T3: Database kya hota hai aur data store kaise hota hai?

Answer: Database ek organized system hai jisme data structured tarike se persist hota hai taaki application restart hone ke baad bhi data delete na ho. Relational databases me data tables (rows aur columns) ke form me store hota hai aur SQL language se query hota hai.

T4: FastAPI kya hai aur ye standard Python frameworks se fast kyun hai?

Answer: FastAPI ek modern, high-performance web framework hai jo Python 3.11+ ke asynchronous features (ASGI) par bana hai. Ye Starlette (web layer) aur Pydantic (data validation layer) use karta hai, jisse iski speed NodeJS aur Go ke barabar hoti hai.

T5: React 18 kya hai aur iska Component-Based Architecture kya hota hai?

Answer: React Facebook dwara banayi gayi JavaScript library hai UI build karne ke liye. Iska component architecture pure UI ko reusable, isolated pieces (Components) me tod deta hai. Jab component ka state change hota hai, to React Virtual DOM ke zariye sirf usi specific part ko re-render karta hai, pura page refresh nahi hota.

T6: Vite kya hai aur traditional Webpack se better kyun hai?

Answer: Vite ek modern frontend build tool hai. Traditional bundlers (Webpack) development me pura code pehle bundle karte the jisme minutes lagte the. Vite native ES Modules (ESM) use karta hai browser me, jisse server instant start hota hai aur code changes Hot Module Replacement (HMR) ke sath millisecond me reflect hote hain.

T7: TypeScript kya hai aur JavaScript ke upar ise kyun prefer kiya gaya?

Answer: TypeScript JavaScript ka statically-typed superset hai. Plain JavaScript me runtime par errors aate hain (jaise `undefined is not a function`). TypeScript compile time par hi data types check kar leta hai, jisse code robust banta hai aur production bugs 40% tak reduce ho jaate hain.

T8: Tailwind CSS kya hai aur standard custom CSS se fast kyun hai?

Answer: Tailwind CSS ek utility-first CSS framework hai jisme pre-built classes (jaise `flex`, `p-4`, `text-blue-500`) milti hain. Alag se lambi CSS files likhne ki zaroorat nahi hoti aur build time par PurgeCSS unused styles ko delete kar deta hai, jisse CSS file size bohot chhota (sirf ~15-20 KB) rehta hai.

T9: SQLite kya hota hai aur file-based database kaise kaam karta hai?

Answer: SQLite ek serverless, self-contained SQL database engine hai jo kisi separate background service ya server ke bina chalta hai. Pura database ek single binary file (`govskill.db`) ke andar store hota hai. Ye ultra-fast hota hai kyunki koi network socket overhead nahi hota.

T10: PostgreSQL kya hota hai aur enterprise production me ye standard kyun hai?

Answer: PostgreSQL ek open-source Object-Relational Database Management System (ORDBMS) hai. Ye massive concurrent transactions, complex joins, JSONB storage, strict ACID compliance, aur enterprise reliability ke liye world-standard mana jaata hai.

T11: ORM (Object-Relational Mapping) kya hai aur SQLAlchemy 2.0 Async ka kya kaam hai?

Answer: ORM ek bridge hai jo database tables ko Python objects ke saath map karta hai. Developer ko raw SQL strings likhne ki zaroorat nahi padti; wo Python code (jaise `User.email == clean_email`) likhta hai aur ORM use safe, parameterized SQL me convert karta hai, jo SQL Injection attacks ko bhi rokta hai.

T12: Database Migrations kya hoti hain aur Alembic ka kya use hai?

Answer: Jab project me naya feature aata hai to database tables me naye columns add karne padte hain. Database migration ek version-controlled script hoti hai jo production me existing data ko delete kiye bina database schema ko safely update karti hai. Alembic Python/SQLAlchemy ka authoritative migration tool hai.

T13: JWT (JSON Web Token) kya hota hai aur stateless auth kaise kaam karta hai?

Answer: JWT ek compact, URL-safe token format hai jisme 3 parts hote hain: Header, Payload (User ID, Role, Expiry), aur Cryptographic Signature. Stateless isliye kehte hain kyunki server ko session memory me store nahi karna padta — server signature verify karke instant authenticate kar leta hai.

T14: Passwords ko plain text me kyun nahi rakhte aur bcrypt kya karta hai?

Answer: Plain text passwords database leak hone par sabke samne expose ho jaate hain. MD5/SHA256 jaise fast hashes rainbow table attacks se crack ho jaate hain. `bcrypt` ek adaptive one-way hashing algorithm hai jisme cryptographic salt aur configurable work factor (cost) hota hai, jisse brute-force attack computationally impossible ho jaata hai.

T15: OCR (Optical Character Recognition) kya hota hai aur Tesseract kaise kaam karta hai?

Answer: OCR ek technology hai jo scanned images ya documents ke pixels ko digital editable text me convert karti hai. Tesseract OCR image ko binarize karta hai, character shapes aur contours ko analyze karta hai, aur trained language models ke against match karke text output nikalta hai.

T16: Google Gemini API kya hai aur Large Language Model (LLM) backend se kaise interact karta hai?

Answer: Google Gemini Google ka multimodal generative AI model hai. Hamara backend REST calls ya official SDK ke zariye context (training modules ya chat history) aur user ka prompt Gemini API ko bhejta hai, aur Gemini grounded conversational response generate karke return karta hai.

T17: REST API kya hota hai aur common HTTP methods kya karte hain?

Answer: REST (Representational State Transfer) ek architectural style hai web services ke liye:

- `GET`: Data read/fetch karne ke liye (jaise list modules).
- `POST`: Naya resource create karne ke liye (jaise register, upload).
- `PUT`: Existing resource ko completely update karne ke liye.
- `DELETE`: Resource delete karne ke liye.

T18: Docker aur Containerization kya hota hai aur Dockerfile ka kya kaam hai?

Answer: Docker ek platform hai jo application aur uski saari dependencies (Python, Tesseract, system libraries) ko ek lightweight container me pack kar deta hai. Isse "*Mere computer par chal raha tha, doosre par nahi chal raha*" wali problem solve ho jaati hai kyunki container har machine par exact same environment me chalta hai.

T19: CI/CD Pipeline aur GitHub Actions kya hota hai?

Answer: CI/CD ka matlab Continuous Integration aur Continuous Deployment hai. GitHub Actions ek automation system hai jo har git push par cloud runner start karta hai, code ko build karta hai, linter (ruff) chalata hai, aur saare 119 backend + 121 frontend tests run karta hai. Agar ek bhi test fail ho to deployment automatically block ho jaata hai.

T20: Cloud Deployment (Render) kya hota hai aur static site vs docker web service me kya antar hai?

Answer: Cloud deployment ka matlab code ko internet par kisi data center ke servers par host karna hai:

- **Static Site (Frontend):** HTML, CSS, aur JS files Render ke global CDN (Content Delivery Network) par deploy hoti hain aur high speed se serve hoti hain.
- **Docker Web Service (Backend):** Ye ek live Linux container hota hai jisme FastAPI server run hota hai aur client ke incoming dynamic API requests ko 24/7 process karta hai.
